

좋습니다. 지금까지 정리된 내용을 보면 이 문서는 단순한 “**AI Coding 도입 계획서**”보다는, **PHY SW의 HW Change → Implementation → Review → Verification → Commercialization** 전체를 **AI와 Engineering Data**로 혁신하는 **PRD**로 잡는 것이 좋습니다.

특히 그룹장 관점에서는 왜 필요한가 → 현재 문제가 무엇인가 → 무엇을 만들 것인가 → 어떻게 검증할 것인가 → 실제 상용화에 어떻게 적용할 것인가 → 성공을 어떻게 측정할 것인가가 명확해야 합니다.

제가 추천하는 PRD 목차는 아래와 같습니다.

PHY SW AI-driven HW Change E2E Engineering PRD

1. Executive Summary

1.1 프로젝트 배경

1.2 문제 정의

1.3 프로젝트 Vision

1.4 핵심 목표

1.5 Expected Business / Engineering Impact

1.6 Executive KPI

한 페이지에서 “왜 이걸 해야 하는가?”를 설명하는 부분입니다.

2. Background & Context

2.1 PHY SW 개발 및 상용화 업무 개요

- Mobile Protocol SW
- Physical Layer SW
- Chipset Bring-up
- Commercialization

2.2 신규 Chipset 개발 Process

- HW Change List
- SW Implementation
- Code Review

- VP
- H-Zebu
- Silicon Bring-up
- Commercial Verification
- ES Release

2.3 HW Change List의 특성

- Chipset당 약 300~400개 Change
- Domain별 분산
- HW/SW dependency
- Register/bit-field 변경
- Design Guide dependency

2.4 현재 개발 환경 및 Tool

- JIRA
- Git/Perforce
- VP
- H-Zebu
- Test Automation
- Design Guide
- CI/CD

3. Problem Statement

여기는 현재의 **Pain Point**를 아주 강하게 정의하는 것이 좋습니다.

3.1 HW Change Review의 한계

- Manual review
- 담당자 assignment
- JIRA comment 기반 communication
- dependency 누락

3.2 HW Change Implementation의 한계

- Implementation 누락
- 잘못된 register configuration
- dependency implementation 누락
- legacy code 이해 부담

3.3 Code Review의 한계

- 초기 단계의 Review

- HW Design Intent와 Code의 연결 부족
- Historical issue 활용 부족

3.4 Pre-silicon Verification의 한계

- VP coverage 제한
- H-Zebu coverage 제한
- SystemC modeling 한계
- Vector coverage 한계
- 모든 HW Change를 simulation으로 검증하기 어려움

3.5 Commercialization 단계의 문제

- Bring-up 이후 defect 발견
- 늦은 defect detection
- 반복적인 debug
- 개발자 resource 증가

3.6 Cross-team Communication 문제

- HW Team ↔ PHY SW Team
- Design Guide 전달
- 변경 의도 전달
- Document version mismatch

4. Current Process & As-Is Workflow

4.1 Current E2E Workflow

```
graph TD; A[HW Change] --> B[Review]; B --> C[Assignment]; C --> D[Implementation]; D --> E[Code Review]; E --> F[VP]; F --> G[H-Zebu]; G --> H[Basic Call]; H --> I[Live Network];
```

↓
Voice
↓
Performance / Functionality
↓
GCF
↓
ES Release

4.2 단계별 Input / Output

각 단계마다:

Stage	Input	Activity	Output	Owner	Problem
-------	-------	----------	--------	-------	---------

4.3 Current Metrics

가능하다면:

- HW Change 수
- Implementation 시간
- Review 시간
- Bring-up defect 수
- Commercial defect 수
- HW Change 관련 issue 수
- 개발자 투입시간

5. Target Vision

여기가 PRD의 핵심 철학입니다.

5.1 Vision

AI-driven HW Change E2E Engineering

HW Change 하나가 생성되는 순간부터 Commercialization까지 모든 Engineering Evidence가 자동으로 연결되는 PHY SW 개발 환경 구축

5.2 핵심 Concept

From

Coverage-driven Verification

To

Evidence-driven Engineering

그리고:

From

Document-driven Development

To

Knowledge-driven Development

From

Manual Change Analysis

To

AI-assisted Impact Analysis

6. Target E2E Workflow

6.1 Target Architecture

```
graph TD; A[HW Change] --> B[HW Change Contract]; B --> C[AI Impact Analysis]; C --> D[AI Risk Assessment]; D --> E[Human Approval]; E --> F[AI Implementation]; F --> G[AI Code Review]; G --> H[Automated Verification]; H --> I[Risk-based VP / H-Zebu]; I --> J[Silicon]; J --> K[Commercial Verification]; K --> L[Knowledge Feedback];
```

HW Change

↓

HW Change Contract

↓

AI Impact Analysis

↓

AI Risk Assessment

↓

Human Approval

↓

AI Implementation

↓

AI Code Review

↓

Automated Verification

↓

Risk-based VP / H-Zebu

↓

Silicon

↓

Commercial Verification

↓

Knowledge Feedback

6.2 HW Change Lifecycle

6.3 Evidence Lifecycle

6.4 Feedback Loop

7. Product Scope

이 프로젝트에서 무엇을 만들 것인가를 정의합니다.

7.1 HW Change Intelligence

- HW Change 분석
- Register diff
- SW Impact Analysis
- Dependency Analysis
- Risk Classification

7.2 Historical HW Change Reconstruction

- 과거 SOC CL 분석
- Register 기반 역추적
- Code Diff
- Change Coupling
- Bug/Issue 연계
- Historical Change DB

7.3 Design Guide Intelligence

- Design Guide structured schema
- Version 관리
- HW-SW Contract
- Design Intent extraction
- Change Traceability

7.4 PHY SW Engineering Knowledge Base

- Architecture
- Code
- Register
- Design Guide
- Historical Change
- Bug
- Verification

7.5 AI Implementation Agent

- Implementation plan
- Code generation
- Patch generation
- Build
- UT
- Static check

7.6 AI Review Agent

- HW ↔ Code comparison
- Design Guide ↔ Code comparison
- Register validation
- Historical issue comparison
- Dependency analysis

7.7 Verification Agent

- Test case recommendation
- UT generation
- VP test selection
- H-Zebu test selection
- Evidence aggregation

7.8 One Dashboard

- HW Change status
- Implementation
- Review
- VP
- H-Zebu
- Silicon
- Commercialization
- JIRA

8. Historical HW Change Knowledge Reconstruction

이 부분은 별도 Chapter로 두는 것을 강력히 추천합니다.

8.1 Why Historical Data Matters

8.2 Historical Data Sources

SOC CL
Git / Perforce
Register DB
Code Diff
Test Case
Bug
JIRA
Design Guide
Silicon Issue

8.3 Historical Change Reconstruction Algorithm

8.4 Register → Code Reverse Trace

8.5 Change Coupling Analysis

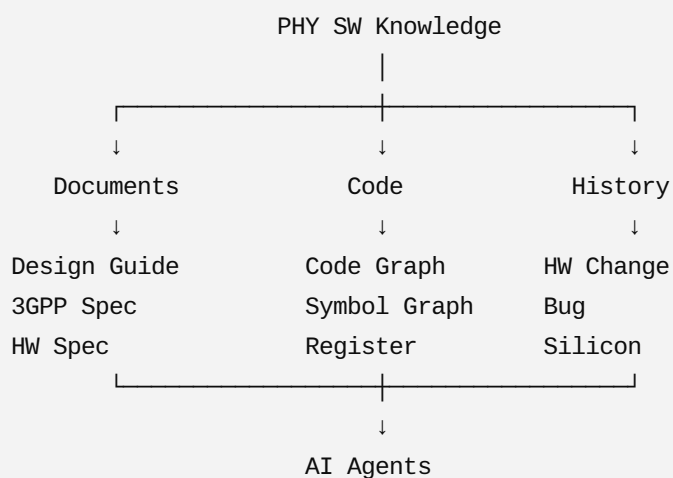
8.6 Historical Dependency Graph

8.7 Confidence Model

- Confirmed
- Probable
- Inferred

9. PHY SW Knowledge Architecture

9.1 Knowledge Architecture



9.2 Machine-generated Knowledge

- AST
- Symbol
- Call Graph

- Register Reference
- Dependency

9.3 Human-curated Knowledge

- Design Intent
- Architecture
- Workaround
- HW limitation
- Historical knowledge

9.4 Knowledge Versioning

9.5 Knowledge Freshness / Drift Detection

9.6 Knowledge Confidence

10. AI Agent Architecture

여기서 **Agentic Coding**을 구체화합니다.

10.1 Agent Architecture

Agent 1 — Change Analyst

HW Change → Impact / Dependency / Risk

Agent 2 — Implementation Agent

Impact → Implementation Plan → Code

Agent 3 — Review Agent

HW / Design / Code → Review

Agent 4 — Verification Agent

Code → Test → Evidence

Agent 5 — Knowledge Agent

Code / Issue / Test → Knowledge Update

11. AI Agentic Coding Workflow

11.1 Planning

11.2 Context Gathering

11.3 Impact Analysis

11.4 Implementation

11.5 Build

11.6 Test

11.7 Review

11.8 Verification

11.9 Evidence Collection

11.10 Human Approval

즉,

**Plan → Context → Implement → Verify → Review → Evidence →
Approve**

를 하나의 loop로 정의합니다.

12. HW Change Review 강화

12.1 Current Review

12.2 Target Review

12.3 AI Review Checklist

HW consistency

Register consistency

Design Guide consistency

SW dependency

Historical issue

Test coverage

Timing / sequencing

Error handling

12.4 Risk-based Review

- Low
- Medium
- High
- Critical

12.5 Human-in-the-loop

AI가 결정하는 것이 아니라:

AI Recommendation → Engineer Approval

구조로 정의합니다.

13. Verification Strategy

이 Chapter가 이번 프로젝트에서는 매우 중요합니다.

13.1 Verification Philosophy

Coverage-driven

에서

Risk / Evidence-driven

으로 전환.

13.2 Verification Layer

```
Static
↓
Contract
↓
UT
↓
VP
↓
H-Zebu
↓
Silicon
```

13.3 Risk-based Verification

Risk	Verification
Low	Static + Contract
Medium	+ UT
High	+ VP
Critical	+ VP + H-Zebu + Human

13.4 VP Coverage Limitation 대응

13.5 H-Zebu Coverage Limitation 대응

13.6 SystemC Modeling Limitation 대응

13.7 Vector Coverage Limitation 대응

13.8 Evidence Aggregation

14. One Dashboard & Traceability

14.1 JIRA-centered Architecture

14.2 HW Change ID

14.3 Traceability

HW Change
↓
Design Guide
↓
SW Code
↓
Review
↓
Test
↓
VP
↓
H-Zebu
↓
Silicon

14.4 Dashboard

14.5 Release Gate

15. KPI & Success Criteria

여기서는 **AI** 성능보다 **Engineering KPI**를 중심으로 잡는 것을 추천합니다.

15.1 Quality KPI

- HW Change escape rate
- Commercial defect rate
- Implementation defect rate
- Review defect detection rate

15.2 Efficiency KPI

- HW Change implementation time
- Review time
- Developer man-hour
- Bring-up debug time

15.3 AI KPI

- Impact Analysis accuracy
- Review recall
- Code generation acceptance rate
- AI recommendation acceptance rate

15.4 Traceability KPI

- HW Change traceability
 - Design Guide traceability
 - Code/Test traceability
 - Evidence completeness
-

16. Pilot Plan

처음부터 **300~400개**를 대상으로 하지 않는다는 원칙을 명시하는 것이 좋습니다.

16.1 Pilot Selection Criteria

16.2 Pilot HW Change 20~30개

16.3 Ground Truth 구축

16.4 AI Impact Analysis Pilot

16.5 AI Review Pilot

16.6 AI Implementation Pilot

16.7 Verification Pilot

16.8 Pilot Success Criteria

17. Implementation Roadmap

제가 지금까지 논의한 내용을 기준으로 하면:

Phase 0 — Foundation

2026 Q3

Historical Data + Knowledge + Ground Truth

↓

Phase 1 — Intelligence

2026 Q3~Q4

AI Impact / Risk / Dependency

↓

Phase 2 — AI Review

2026 Q4

HW ↔ Design ↔ Code Review

↓

Phase 3 — AI Implementation

2026 Q4~2027 Q1

Low-risk HW Change 자동 구현



Phase 4 — Verification Integration

2027

UT + VP + H-Zebu



Phase 5 — E2E Agentic Engineering

2027~2028

HW Change → Commercialization

18. System Architecture

18.1 Overall Architecture

18.2 Data Layer

- Git / Perforce
- JIRA
- Register DB
- Design Guide
- Test DB
- VP
- H-Zebu

18.3 Knowledge Layer

- Vector DB
- Code Index
- Knowledge Graph
- Metadata DB

18.4 AI Layer

- LLM
- RAG
- Agents
- MCP / Tools

18.5 Automation Layer

- CI/CD

- Build
- Test
- VP
- H-Zebu
- JIRA

19. Security & Governance

PHY SW는 일반적인 Coding Agent와 달리 이 부분이 중요합니다.

19.1 Source Code Security

19.2 LLM Data Security

19.3 AI-generated Code Policy

19.4 Human Approval Policy

19.5 Audit Trail

19.6 Knowledge Access Control

19.7 Hallucination / False Positive Management

20. Risk & Mitigation

Risk	Mitigation
AI Hallucination	Evidence-based output
Legacy Code complexity	Code Graph + Knowledge
Historical data 부족	Reconstruction
VP Coverage 부족	Risk-based verification
Design Guide 품질	HW Change Contract
Knowledge drift	CI-based update
AI code 오류	Human approval

Risk	Mitigation
Engineer resistance	Pilot + KPI
Agent autonomy 과다	Permission / Gate

21. Out of Scope

이것도 PRD에서 반드시 넣는 것을 추천합니다.

예를 들어 초기 단계에서는:

- 모든 HW Change 100% 자동 구현
- AI가 최종 Code Review 승인
- AI가 Silicon issue를 독립적으로 판단
- AI가 HW design intent를 임의로 결정
- VP coverage 자체를 무조건 확대

등은 **Out of Scope**로 명확하게 정의합니다.

22. Deliverables

최종적으로 무엇을 만들어야 하는지를 명확히 합니다.

D1. HW Change Contract

D2. Historical HW Change DB

D3. PHY SW Engineering Knowledge Base

D4. AI Change Analyst

D5. AI Implementation Agent

D6. AI Review Agent

D7. Verification Agent

D8. One Dashboard

D9. E2E Traceability System

23. Appendix

A. HW Change Contract Schema

B. JIRA Issue Schema

C. Knowledge Card Schema

D. AI Review Rubric

E. Risk Classification Rubric

F. Historical Change Reconstruction Rules

G. AI Agent Prompt / Tool Specification

H. KPI Definition

I. Pilot Dataset

J. Benchmarking Reference

제가 특히 추천하는 **PRD** 구조

전체를 크게 **5개의 Part**로 묶으면 경영진/개발자 모두 읽기 좋습니다.

PART I

WHY

1. Executive Summary
2. Background
3. Problem Statement
4. As-Is Process

PART II

WHAT

5. Target Vision
6. Target E2E Workflow
7. Product Scope
8. Key Deliverables

PART III

HOW

9. Historical Change Intelligence
10. Knowledge Architecture
11. AI Agent Architecture
12. AI Coding Workflow
13. AI Review
14. Verification Strategy
15. One Dashboard

PART IV EXECUTION

16. Pilot
17. KPI
18. Roadmap
19. System Architecture
20. Security / Governance
21. Risk

PART V APPENDIX

22. Schema
23. Rubric
24. Agent Specification
25. Benchmark

그리고 이 **PRD**에서 가장 중요한 문장 하나를 꼽는다면

“The goal is not to automate coding. The goal is to establish an evidence-driven, traceable, AI-assisted E2E engineering system for every HW Change.”

이렇게 잡는 것이 좋겠습니다.

그래야 **AI Coding → AI Agent → VP → JIRA Dashboard**가 각각 따로 노는 프로젝트가 아니라, **“PHY SW의 HW Change 대응 비용과 상용화 defect를 구조적으로 줄이는 하나의 제품”**으로 묶입니다.

다음 단계에서는 이 목차를 바탕으로 실제 **PRD 초안(Executive Summary부터 Problem Statement, Target Architecture, KPI, Roadmap까지)**을 작성하면 됩니다. 특히 그룹장 관점에서 상사에게 보고할 수 있도록 **“Why now / Expected impact / 올해 반드시 달성할 3가지”**를 앞부분에 강하게 넣는 형태가 적합합니다.